SCIKIT-LEARN

Complete Guide to Machine Learning in Python

Beginner-to-intermediate reference

Concepts • Algorithms • Preprocessing • Training • Evaluation • Pipelines • Projects

A practical learning document with explanations, Python examples, terminology, and a recommended study roadmap.

Contents

- 1. What Scikit-learn is and why it is used
- 2. Installation and the Python ML ecosystem
- 3. Types of machine learning
- 4. Main algorithm categories
- 5. Classification, regression, and clustering
- 6. Data preprocessing and feature engineering
- 7. Training/testing splits and model evaluation
- 8. Feature selection, dimensionality reduction, and ensembles
- 9. Hyperparameter tuning and pipelines
- 10. Important APIs, NLP, neural networks, and model persistence
- 11. End-to-end workflow, terminology, projects, and roadmap

Note: Code examples are educational starting points. Real projects need suitable datasets, validation, and careful preprocessing.

1. What is Scikit-learn?

Scikit-learn (imported in Python as `sklearn`) is an open-source Python library for traditional machine learning. It provides consistent tools for preparing data, training models, making predictions, selecting models, and evaluating performance.

Rather than implementing common algorithms from scratch, you can use tested implementations and focus on understanding the data and the problem.

Common uses

- Classification: predict a category, such as spam or not spam.
- Regression: predict a number, such as a house price.
- Clustering: discover groups in data, such as customer segments.
- Preprocessing: handle missing values, encode categories, and scale features.
- Model selection: compare models and tune their settings.
- Evaluation: measure how well a model performs on unseen data.

2. Installation and the Python ecosystem

```
python -m pip install scikit-learn numpy pandas scipy matplotlib seaborn jupyter
```

The package is installed as `scikit-learn`, but imported as `sklearn`.

```
import sklearn
print(sklearn.__version__)
```

Library	Typical role
NumPy	Arrays and numerical calculations
Pandas	Loading, cleaning, and manipulating tabular data
SciPy	Scientific and numerical computing
Matplotlib	Charts and visualizations

Library	Typical role
Seaborn	Statistical visualizations
Scikit-learn	Machine learning algorithms and workflows

3. Types of machine learning

Supervised learning

The training data contains input features and known target labels or values. The model learns a relationship that can be used to predict targets for new examples.

- Classification predicts categories.
- Regression predicts continuous numerical values.

Unsupervised learning

The data has no supplied target labels. The algorithm looks for structure, groups, or lower-dimensional representations.

- Clustering groups similar observations.
- Dimensionality reduction represents data using fewer features.

Reinforcement learning

An agent learns by interacting with an environment and receiving rewards or penalties. Scikit-learn is not a general-purpose reinforcement-learning framework; specialized libraries are usually used.

4. Main categories of sklearn tools

Category	Purpose	Examples
Classification	Predict classes	LogisticRegression, SVC, RandomForestClassifier
Regression	Predict numbers	LinearRegression, RandomForestRegressor, SVR
Clustering	Group observations	KMeans, DBSCAN, AgglomerativeClustering
Dimensionality reduction	Reduce feature dimensions	PCA, TruncatedSVD
Ensemble learning	Combine multiple estimators	Random Forest, Gradient Boosting
Preprocessing	Prepare input data	SimpleImputer, StandardScaler, OneHotEncoder
Model selection	Split, validate, and tune	train_test_split, GridSearchCV
Feature selection	Choose useful input columns	SelectKBest, RFE

5. Classification

Classification predicts a discrete class or category. Examples include spam detection, fraud screening, customer churn prediction, and image-category recognition.

Algorithm	How it works (high level)
Logistic Regression	Models class probabilities; despite its name, commonly used for classification.
Decision Tree	Uses a sequence of feature-based decision rules.
Random Forest	Combines predictions from many decision trees.
K-Nearest Neighbors (KNN)	Predicts using nearby training observations.
Support Vector Machine (SVM)	Finds a boundary that separates classes.
Naive Bayes	Uses probability and a conditional-independence assumption.
Gradient Boosting	Builds models sequentially to correct earlier errors.

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB

messages = [
    "Win a free mobile",
    "Meeting at 10 AM",
    "Claim your free prize",
    "Project meeting tomorrow",
    "You won a lottery",
    "Please send the report"
]
labels = ["spam", "not_spam", "spam",
          "not_spam", "spam", "not_spam"]

vectorizer = CountVectorizer()
X = vectorizer.fit_transform(messages)

X_train, X_test, y_train, y_test = train_test_split(
    X, labels, test_size=0.33, random_state=42,
    stratify=labels
)

model = MultinomialNB()
model.fit(X_train, y_train)

new_message = vectorizer.transform(["Win a free prize"])
print(model.predict(new_message))
```

This tiny example demonstrates the workflow; reliable spam detection requires a much larger, representative dataset.

6. Regression

Regression predicts numerical values, such as prices, sales, demand, or temperature.

Algorithm	Description
Linear Regression	Fits a linear relationship between features and a numerical target.
Ridge	Linear regression with L2 regularization.
Lasso	Linear regression with L1 regularization; can shrink some coefficients to zero.
Decision Tree Regressor	Predicts using decision rules.
Random Forest Regressor	Averages predictions from multiple trees.
SVR	Support Vector Regression.
Gradient Boosting Regressor	Combines sequentially improved regression trees.

```
import numpy as np
from sklearn.linear_model import LinearRegression
```

```
# Area in square feet
X = np.array([[1000], [1500], [2000], [2500], [3000]])

# Price in lakhs
y = np.array([30, 45, 60, 75, 90])

model = LinearRegression()
model.fit(X, y)

predicted_price = model.predict([[1800]])
print(predicted_price)  # approximately [54.]
```

7. Clustering

Clustering is an unsupervised learning technique that groups similar observations without requiring pre-existing labels.

Algorithm	Description
K-Means	Partitions observations into a chosen number of clusters.
DBSCAN	Finds dense groups and can identify noise points.
Agglomerative Clustering	Builds a hierarchy of merged clusters.
Mean Shift	Finds dense regions in feature space.

```
import numpy as np
from sklearn.cluster import KMeans

X = np.array([
    [20, 2000], [22, 2500], [25, 3000],
    [40, 8000], [42, 8500], [45, 9000]
])

model = KMeans(n_clusters=2, random_state=42, n_init=10)
model.fit(X)

print(model.labels_)
```

The example uses age and spending as two features. In real applications, scale features before K-Means because distances are sensitive to feature magnitude. Cluster IDs (0, 1, etc.) do not have an inherent business meaning.

8. Data preprocessing

Real datasets may contain missing values, inconsistent categories, different numerical scales, and other issues. Preprocessing converts raw data into a form suitable for an algorithm.

8.1 Missing values: SimpleImputer

```
import numpy as np
from sklearn.impute import SimpleImputer

X = np.array([
    [10, 20],
    [15, np.nan],
    [20, 30],
    [np.nan, 40]
])

imputer = SimpleImputer(strategy="mean")
X_filled = imputer.fit_transform(X)
print(X_filled)
```

Common strategies include `mean`, `median`, `most_frequent`, and `constant`.

8.2 Feature scaling

Scaling puts numerical features on comparable scales. It is especially important for distance-based and gradient-based methods, such as KNN, SVM, and many linear models.

```
from sklearn.preprocessing import StandardScaler, MinMaxScaler

X = [[25, 30000], [30, 50000], [35, 80000]]

standard = StandardScaler()
X_standard = standard.fit_transform(X)

minmax = MinMaxScaler()
X_minmax = minmax.fit_transform(X)
```

StandardScaler transforms features to approximately zero mean and unit variance. MinMaxScaler normally maps each feature into the range 0 to 1.

8.3 Encoding categories

```
from sklearn.preprocessing import OneHotEncoder

data = [["Red"], ["Blue"], ["Green"], ["Red"]]

encoder = OneHotEncoder(sparse_output=False)
encoded = encoder.fit_transform(data)
print(encoded)
```

OneHotEncoder is appropriate for nominal categories without a natural order. LabelEncoder is mainly intended for encoding target labels, not as a default way to encode unordered input features.

9. Splitting training and testing data

Evaluate a model on data that was not used to train it. A common starting point is 80% training data and 20% test data, although the best split depends on the dataset and task.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)
```

- **X:** input features.
- **y:** target values or labels.
- **test_size=0.2:** reserve 20% for testing.
- **random_state=42:** make the split reproducible.
- **stratify=y:** for classification, preserve class proportions where appropriate.

10. Model evaluation

Classification metrics

Metric	Meaning
Accuracy	Fraction of predictions that are correct.
Precision	Of predicted positives, how many were truly positive?
Recall	Of actual positives, how many were found?
F1 score	Harmonic mean of precision and recall.
Confusion matrix	Counts of correct and incorrect class predictions.
ROC-AUC	Measures ranking quality across classification thresholds.

```
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, confusion_matrix
```

```
)

y_true = [1, 0, 1, 1, 0, 1]
y_pred = [1, 0, 1, 0, 0, 1]

print("Accuracy:", accuracy_score(y_true, y_pred))
print("Precision:", precision_score(y_true, y_pred))
print("Recall:", recall_score(y_true, y_pred))
print("F1:", f1_score(y_true, y_pred))
print(confusion_matrix(y_true, y_pred))
```

Regression metrics

Metric	Meaning
MAE	Average absolute prediction error.
MSE	Average squared prediction error.
RMSE	Square root of MSE; expressed in target units.
R^2	Proportion of target variance explained relative to a baseline.

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

y_true = [100, 200, 300, 400]
y_pred = [110, 190, 310, 380]

print("MAE:", mean_absolute_error(y_true, y_pred))
print("MSE:", mean_squared_error(y_true, y_pred))
print("RMSE:", mean_squared_error(y_true, y_pred) ** 0.5)
print("R2:", r2_score(y_true, y_pred))
```

11. Feature selection

Feature selection identifies which input columns are useful for a particular prediction task. It can reduce noise, simplify a model, and sometimes improve generalization.

Tool	Purpose
SelectKBest	Selects the K highest-scoring features using a scoring function.
SelectFromModel	Selects features based on importance or coefficients from an estimator.
RFE	Recursively removes less useful features.
VarianceThreshold	Removes features with low variance.

```
from sklearn.feature_selection import SelectKBest, f_regression

selector = SelectKBest(score_func=f_regression, k=2)
X_selected = selector.fit_transform(X, y)
```

12. Dimensionality reduction

Dimensionality reduction represents data using fewer features while retaining useful structure. It can help with visualization, noise reduction, and computational efficiency.

Method	Purpose
PCA	Projects data onto directions of high variance.
TruncatedSVD	Reduces dimensions, often useful for sparse matrices.
NMF	Factorizes non-negative data into lower-dimensional components.
t-SNE	Creates low-dimensional visualizations of high-dimensional data.

Method	Purpose
Isomap	Nonlinear manifold learning and dimensionality reduction.

```
from sklearn.decomposition import PCA

pca = PCA(n_components=2)
X_reduced = pca.fit_transform(X)
```

13. Ensemble learning

Ensemble learning combines multiple estimators to produce a prediction. It can improve predictive performance or robustness compared with an individual model.

- Random Forest: combines many decision trees.
- Gradient Boosting: builds estimators sequentially to correct errors.
- AdaBoost: emphasizes observations that earlier estimators misclassified.
- Bagging: trains estimators on sampled subsets and combines their predictions.
- Voting: combines predictions from multiple different models.
- Stacking: trains a final estimator using predictions from base estimators.

```
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)
model.fit(X_train, y_train)
predictions = model.predict(X_test)
```

14. Hyperparameter tuning and cross-validation

Hyperparameters are settings chosen before or during model selection; they are not the learned parameters fitted from training data. Tuning searches for settings that perform well under a validation strategy.

GridSearchCV

```
from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(random_state=42)

params = {
    "n_estimators": [50, 100, 200],
    "max_depth": [3, 5, 10]
}

grid = GridSearchCV(
    model,
    params,
    cv=5,
    scoring="accuracy"
)
grid.fit(X_train, y_train)

print(grid.best_params_)
print(grid.best_score_)
```

Other model-selection tools

- RandomizedSearchCV: evaluates a chosen number of sampled parameter combinations.
- cross_val_score: evaluates an estimator using cross-validation.
- cross_validate: evaluates multiple metrics and returns more detailed results.

- learning_curve: studies performance as training-set size changes.

15. Pipelines

A Pipeline chains preprocessing and a model into one estimator. It helps keep training and prediction steps consistent and reduces the risk of preprocessing leakage.

```
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="mean")),
    ("scaler", StandardScaler()),
    ("model", LogisticRegression())
])

pipeline.fit(X_train, y_train)
predictions = pipeline.predict(X_test)
```

During fitting, each step learns from the training data. When predicting, the same fitted transformations are applied before the model produces output.

16. Important sklearn methods and attributes

Method / attribute	Meaning
fit(X, y)	Fit an estimator to training data.
predict(X)	Predict target values or classes.
fit_predict(X)	Fit and return cluster labels for supported estimators.
transform(X)	Apply a learned transformation.
fit_transform(X)	Fit a transformer and transform the data.
predict_proba(X)	Return class probabilities when supported.
score(X, y)	Return an estimator-specific default score.
get_params()	Get estimator parameters.
set_params()	Set estimator parameters.
classes_	Class labels learned by a classifier, when supported.
coef_	Learned coefficients for supported linear estimators.
feature_importances_	Feature importance for supported estimators.
n_features_in_	Number of input features seen during fitting.

Not every estimator implements every method or attribute. Check the relevant estimator documentation.

17. Text processing and NLP

Scikit-learn provides feature extraction tools that convert text into numerical representations for conventional machine learning models.

Tool	Purpose
CountVectorizer	Converts documents into word-count features.
TfidfVectorizer	Converts documents into TF-IDF features.

Tool	Purpose
TfidfTransformer	Transforms count features into TF-IDF features.
HashingVectorizer	Maps tokens into a fixed-size feature space using hashing.

```
from sklearn.feature_extraction.text import TfidfVectorizer

documents = [
    "Machine learning is interesting",
    "Python is useful for machine learning",
    "I love programming"
]

vectorizer = TfidfVectorizer()
matrix = vectorizer.fit_transform(documents)

print(vectorizer.get_feature_names_out())
print(matrix.toarray())
```

18. Neural networks in sklearn

Scikit-learn includes basic feed-forward neural network estimators based on Multi-Layer Perceptrons (MLPs).

- MLPClassifier: classification using an MLP.
- MLPRegressor: regression using an MLP.

```
from sklearn.neural_network import MLPClassifier

model = MLPClassifier(
    hidden_layer_sizes=(100, 50),
    max_iter=1000,
    random_state=42
)

model.fit(X_train, y_train)
predictions = model.predict(X_test)
```

For complex deep learning, GPU training, modern computer vision, and large language models, PyTorch and TensorFlow are generally more suitable.

19. Saving and loading a model

A trained model can be persisted and reused, rather than retrained every time an application starts.

```
import joblib

# Save
joblib.dump(model, "trained_model.pkl")

# Load
loaded_model = joblib.load("trained_model.pkl")

# Predict
prediction = loaded_model.predict(X_new)
```

Only load pickle/joblib files from trusted sources. Keep compatible versions of Python and dependencies, and consider model persistence security before deploying.

20. Important sklearn modules

Module	What it contains
sklearn.linear_model	Linear and logistic models
sklearn.tree	Decision trees

Module	What it contains
sklearn.ensemble	Random forests, boosting, bagging, voting
sklearn.cluster	Clustering algorithms
sklearn.svm	Support vector machines
sklearn.neighbors	Nearest-neighbor algorithms
sklearn.naive_bayes	Naive Bayes classifiers
sklearn.preprocessing	Scaling and encoding
sklearn.impute	Missing-value tools
sklearn.model_selection	Data splitting, validation, and tuning
sklearn.metrics	Evaluation metrics
sklearn.decomposition	Dimensionality reduction
sklearn.feature_selection	Feature selection
sklearn.pipeline	Estimator pipelines
sklearn.datasets	Built-in sample datasets and data generators
sklearn.feature_extraction	Text and other feature extraction

21. Machine learning terminology

Term	Explanation
Dataset	A collection of observations.
Sample	One observation or row.
Feature	An input variable or column.
Target	The value or label the model learns to predict.
Training	Learning patterns from data.
Testing	Evaluating with held-out data.
Model	A fitted estimator that maps inputs to outputs.
Algorithm	The method used to learn from data.
Parameters	Values learned during fitting.
Hyperparameters	Settings chosen outside the model's fitted parameters.
Overfitting	Learning training-specific patterns that do not generalize.
Underfitting	Failing to learn important patterns.
Generalization	Performance on new, unseen observations.
Cross-validation	Repeated evaluation across different data splits.
Data leakage	Using information during training that would not legitimately be available.
Regularization	Constraining model complexity to help control overfitting.

22. Complete machine learning workflow

- **Collect data:** obtain CSV, Excel, database, API, or other source data.
- **Explore data:** inspect distributions, relationships, missing values, and class balance.
- **Clean and preprocess:** handle missing values, encode categories, and scale where appropriate.
- **Split data:** create training and test sets; use validation or cross-validation for model selection.
- **Choose an algorithm:** select an approach suited to the task and data.
- **Train:** call fit() on training data.
- **Evaluate:** use metrics appropriate to the problem and its costs.
- **Tune:** improve hyperparameters using validation data or cross-validation.
- **Deploy and monitor:** integrate the model into an application and monitor real-world performance.

23. Practical project ideas

Project	Possible algorithms	Level
Student marks prediction	Linear Regression	Beginner
Email spam detection	Naive Bayes	Beginner
Customer segmentation	K-Means	Intermediate
House price prediction	Linear Regression, Random Forest	Intermediate
Credit card fraud detection	Logistic Regression, Random Forest	Advanced
Product recommendation	Nearest neighbors and similarity methods	Advanced

For fraud detection, pay special attention to class imbalance, precision, recall, and the cost of false positives and false negatives.

24. Recommended learning roadmap

Stage	Topics	Suggested time
1	Python fundamentals	1–2 weeks
2	NumPy	1 week
3	Pandas	1–2 weeks
4	Matplotlib and Seaborn	1 week
5	Statistics and probability	1–2 weeks
6	Machine learning fundamentals	1 week
7	Preprocessing with sklearn	1 week
8	Classification and regression	2 weeks
9	Clustering and dimensionality reduction	1 week
10	Evaluation and hyperparameter tuning	1–2 weeks
11	Real-world projects	2–4 weeks

The durations are estimates. Spend more time practising with real datasets rather than trying to finish every topic quickly.

25. What sklearn is not designed for

Task	Commonly used technology
Traditional machine learning	Scikit-learn
Deep learning	PyTorch or TensorFlow
Large language models	PyTorch and Transformer libraries
Generative AI applications	Specialized LLM frameworks and model APIs
Deep-learning computer vision	PyTorch or TensorFlow
Reinforcement learning	Gymnasium with reinforcement-learning libraries

Scikit-learn remains useful alongside these technologies for preprocessing, classical models, baselines, and evaluation.

26. Official documentation and references

- Official website: https://scikit-learn.org/stable/
- User Guide: https://scikit-learn.org/stable/user_guide.html
- Getting Started: https://scikit-learn.org/stable/getting_started.html
- Examples: https://scikit-learn.org/stable/auto_examples/index.html
- Tutorials: https://scikit-learn.org/stable/tutorial/index.html

27. Final recommendations

Start by understanding the common estimator workflow: prepare X and y, split the data, fit a model, predict, and evaluate. Then practise preprocessing, classification, regression, clustering, cross-validation, and pipelines. Build small projects as you learn.

Core idea: Pandas and NumPy help prepare and represent your data; Scikit-learn helps you train and evaluate traditional machine learning models.